FOAI (FOUNDATIONS OF ARTIFICIAL INTELLIGENCE) EXAM & PROJECT GUIDE

AERON: Autonomous Aerial Flight Planner

Complete Technical Guide, AI Algorithms Breakdown & Oral Defense Notes

Subject: FOAI / AI Systems
Vehicle: Roadster Aero EVTOL

💡 The Core Exam Question: Did We Use AO* or Rule-Based AI?

Direct Answer to give your Examiner: We designed a **Hybrid AI Architecture** combining **BOTH** techniques in complementary roles:

- **AO* Heuristic Search** is our **Primary Graph Planning Engine**. It operates on a formal **AND-OR Graph** to evaluate complex composite corridors, propagate costs bottom-up, and dynamically switch branches when conditions change.
- **Rule-Based AI** is our **Deterministic Safety Auditor**. It evaluates aerospace hard rules (15% battery reserve floor, cylinder no-fly zones, obstacle blocks).

How they work together: The Rule-Based AI validates every corridor and feeds directly into the *AND conjunct check nodes* of the AO* graph. If any safety rule fails, that conjunct is marked `FAIL (Cost = ∞)`, which triggers the **AO* search to immediately prune that corridor and backtrack** to select an alternative flight path or divert to an emergency landing vertipad!

1. What Is This Project About? (Simple Explanation)

Standard navigation systems (like Google Maps or traditional A* search) are designed for ground cars: they simply find the shortest route between two coordinates. In **urban aerial transport (flying cars / EVTOLs)**, this approach is dangerous because vehicles cannot pause mid-air if their battery drops or wind speeds spike.

AERON introduces "AI That Imagines Before It Acts":

- **Simultaneous Safety Checks:** Reaching the next waypoint requires navigation *AND* clearance checks (battery reserve ≥ 15%, no storm wind shear, no prohibited airspace).
- **Autonomous Vertipad Diversion:** If battery is degraded (e.g. 32%), the AI does not crash. It realizes the destination is unreachable and safely diverts to the nearest certified emergency vertipad.
- **Pre-Flight Counterfactual Reasoning:** Simulates 6 "what-if" scenarios before take-off to verify resilience against storms, drone swarms, and pad closures.

2. Complete Technology Stack

Layer	Technology Used	Role & Contribution
Core Search Algorithm	AO* Heuristic Search (Pure Python)	Navigates AND-OR graph, propagates bottom-up costs, marks best partial tree.
Safety Reasoning	Deterministic Rule-Based Engine	Evaluates hard rules (≥15% reserve, no-fly zones) & soft costs (turbulence).
Aerodynamics & Physics	Kinematic Flight & Drag Model	Computes drag (v^3), climb power, 28% descent regen, and wind factor.
Backend Web Service	Python 3.9+, FastAPI, Pydantic v2	Provides high-performance REST APIs for mission planning and dynamic replanning.
Frontend & 3D Graphics	React 18, TypeScript, Three.js	Renders real-time 3D Metropolis airspace, procedural skyscrapers, and HUD.
Verification & Baselines	Pytest, Baseline A* & Greedy BFS	10 passing tests; empirical evaluation across 5 seeded benchmark missions.

FOAI CORE CONCEPT #1

AO* Algorithm & AND-OR Hypergraphs

Why standard A* is insufficient and how AO* handles decomposed flight constraints

1. Why Standard A* Search Fails in Urban Airspace

Standard search algorithms (like Dijkstra, A*, or Greedy BFS) assume **serial transitions**: an action transitions the agent from state *u* to state *v*.

In autonomous flight, moving along a corridor involves **decomposed subgoals and contingent constraints**. To take corridor *A* → *B*, reaching waypoint *B* is only valid if environmental clearance simultaneously succeeds:

```
Action(Waypoint_i → Waypoint_j) &implies; Waypoint_j  ∧  ClearanceCheck(i, j)
```

OR Nodes (Alternative Choices)

Waypoints represent **OR nodes** where the planner chooses among departing corridors (e.g., choice between North Skyway, Waterfront Low Corridor, or Central Core).

AND Nodes (Conjunctive Hyperarcs)

Selecting a corridor connects to **both** the target waypoint **AND** a safety clearance node. If remaining battery < 15% or corridor is blocked, that conjunct fails!

2. How the AO* Algorithm Works (Step-by-Step)

1. **Tree Initialization:** Search begins at the start vertiport with an admissible heuristic estimate to the goal.
2. **Best Partial Tree Traversal:** Follows currently marked best hyperarcs from root to identify unexpanded leaf nodes.
3. **Node Expansion:** Selects the most promising leaf node, expanding candidate hyperarcs and their conjunct children.
4. **Bottom-Up Cost Propagation:** Calculates cost for each hyperarc:

   ```
   cost(A) = c(A) + ∑_{m ∈ Targets(A)} f(m)  |  f(n) = min_{A ∈ Hyperarcs(n)} cost(A)
   ```

   If any conjunct target in an AND branch is infeasible (∞), that entire hyperarc becomes ∞.
5. **Automatic Branch Switching:** If an expanded branch reveals unexpected headwind drag, obstacle blockage, or high energy consumption, ancestor node values update, immediately switching the marked action to an alternative corridor.
6. **Termination:** Concludes when the root node is marked `SOLVED` (success) or `INFEASIBLE` (no safe path).

3. Multi-Objective Admissible Heuristic h(n)

A heuristic is **admissible** if it never overestimates true remaining cost. AERON computes straight-line 3D Euclidean distance at maximum velocity (60 m/s) with ideal minimum cruise power (15 kW):

```
t_ideal = Distance_3D / v_max  |  E_ideal = (P_min × t_ideal) / 3600  |  h(n) = w_time×(t_ideal/10) + w_energy×(4×E_ideal)
```

Because any realistic flight has wind resistance, altitude climbs, and maneuvers, actual time and energy will always be greater than or equal to this estimate ($h(n) \le h^*(n)$), guaranteeing global optimality.

FOAI CORE CONCEPT #2

Rule-Based AI & Safety Auditing

Deterministic constraints, autonomous emergency vertipad divert, and benchmark comparisons

1. What Role Does Rule-Based AI Play?

While AO* searches the combinatorial graph, the **Rule-Based AI Layer** acts as a strict aerospace safety auditor. It ensures that no plan is cleared if it violates critical flight regulations.

2. Hard Rules vs. Soft Costs

Constraint Type	Rule Definition	System Reaction
Hard Rule 1 **15% Battery Reserve**	Final landing state must preserve ≥ 15.0% Emergency Reserve Floor.	If SoC < 15%, clearance conjunct fails (Cost = ∞). Planner prunes branch.
Hard Rule 2 **No-Fly Airspaces**	Flight leg must not intersect 3D cylindrical restricted zones (Civic Sanctuary).	Corridor is immediately eliminated from candidate actions.
Hard Rule 3 **Corridor Blockage**	Corridor must be free of airborne obstacles or drone swarms.	Blocked corridor marked infeasible; AO* selects alternative bypass.
Hard Rule 4 **Pad Availability**	Destination vertipad must be certified and open for landing.	If destination pad closes, terminal goal shifts to emergency vertipad.
Soft Preference 1 **Weather Drag**	Avoid coastal crosswinds and storm gale turbulence cells.	Adds soft cost multiplier (up to 2.4×) to edge weights.
Soft Preference 2 **Mission Priority**	User weights for BALANCED, SAFEST, LOWEST_ENERGY, or SHORTEST_TIME.	Dynamically scales weights between time, energy (kWh), and risk.

3. The Autonomous Diversion Showcase (Low Battery Scenario)

Example: What happens when the user sets initial battery to 32%?
1. Reaching the primary destination (Neo-Pinnacle Vertiport) requires ~22.4% battery.
2. Starting with 32% leaves only 9.6% SoC, **breaching the mandatory 15% reserve floor**.
3. Standard A* fails or runs out of battery mid-air.
4. **AERON's AO* search** marks the primary destination branch as infeasible, explores contingency branches, and identifies that *Industrial Tech Vertipad Alpha* is reachable with **21.5% remaining SoC**.
5. The system safely diverts, displays a `DIVERTED` status badge, and explains its causal rationale!

4. Empirical Benchmark Results (AO* vs. Baselines)

Benchmark Scenario	AO* Result	Standard A* Result	Greedy Search Result
B1: Nominal Transit	✓ Solved (1.8 ms)	✓ Solved (0.9 ms)	✓ Solved (0.8 ms)
B2: High-Speed Skyline	✓ Solved (1.7 ms)	✓ Solved (0.8 ms)	✓ Solved (0.7 ms)
B3: Severe Weather Gale	✓ Solved (1.9 ms)	✓ Solved (1.0 ms)	✓ Solved (0.9 ms)
B4: Dense Constraints	✓ Solved (2.4 ms)	✓ Solved (1.3 ms)	× **Failed (Trapped in loop)**
B5: Critical Battery Reserve	✓ **Solved (Diverts to Pad)**	× **Failed (Infeasible)**	× **Failed (Infeasible)**

AEROSPACE PHYSICS & REASONING

Aerodynamic Power Model & Counterfactual Reasoning

Realistic physics, 6 pre-flight 'what-if' simulations, and in-flight replanning

1. Realistic Aerodynamic Power Model

Aerodynamic Drag Power

```
P_drag = 0.5 × ρ × C_d × A × v_air³
```

Scales with the **cube of airspeed** relative to headwind. Mass = 1850 kg, A = 2.1 m², C_d = 0.22, ρ = 1.225 kg/m³.

Gravitational Potential & Regen

```
P_climb = (m × g × Δz) / Δt
```

Climbing draws potential energy; descending captures **28% regenerative energy** back to battery (capped at -12 kW).

2. Pre-Flight Counterfactual Reasoning (6 "What-If" Scenarios)

Before taking off, AERON simulates 6 alternative hypothetical realities to verify resilience:

- **CF 1 (Low Battery - 34% SoC):** Tests autonomous divert to certified Industrial Tech Vertipad Alpha.
- **CF 2 (Corridor Blocked):** Primary skyway blocked by drone swarm; tests detour via Waterfront Bypass.
- **CF 3 (Storm Gale):** Coastal crosswind reaches 32 m/s with 2.4× turbulence drag penalty; ascends to ridge corridor.
- **CF 4 (Destination Pad Closed):** Neo-Pinnacle vertipad closes; shifts terminal goal to Substation Vertipad.
- **CF 5 (Prohibited Airspace Active):** Civic Sanctuary cylinder activates; verifies perimeter corridor avoidance.
- **CF 6 (Zero-Risk Policy):** Risk tolerance tightened to 0.05; selects ultra-conservative transit channels.

3. In-Flight Dynamic Replanning Without Teleportation

While the vehicle is flying in the 3D simulator, disturbances can be injected live (e.g. drone blockage, sudden battery drop).

Non-Teleportation Splicing: When a disturbance occurs at current waypoint W_{curr} with remaining battery SoC_{curr}, AO* re-plans from that exact state and splices the traversed history with the newly planned trajectory:

```
Trajectory = [W_start, ..., W_{curr-1}]_traversed ∪ [W_curr, ..., W_term]_new
```

4. Software Architecture Breakdown

```
FOAI OG/
├── backend/
|  ├── app/algorithms/ao_star.py # Formal AO* Search implementation from scratch
|  ├── app/algorithms/graph.py # AND-OR Flight Graph & Hyperarc Builder
|  ├── app/algorithms/heuristics.py # Multi-objective admissible heuristic h(n)
|  ├── app/algorithms/baseline_planners.py# Baseline A* and Greedy Best-First Search
|  ├── app/simulation/physics.py # Drag power (v³), climb power, 28% descent regen
|  ├── app/simulation/environment.py # 3D city airspace, buildings, restricted zones, weather
|  ├── app/reasoning/counterfactuals.py # Deterministic 6-scenario what-if engine
|  ├── app/reasoning/safety_rules.py # Rule-based hard/soft safety auditor
|  ├── app/reasoning/dynamic_replanning.py# In-flight disturbance injection & trajectory splicing
|  └── app/api/endpoints.py # FastAPI REST endpoints (/plan, /replan, /counterfactuals)
└── frontend/
├── src/components/MetropolisScene.tsx # Three.js 3D city airspace & Catmull-Rom spline
├── src/pages/FlightSimulatorPage.tsx # 3D simulator with live disturbance injection
├── src/pages/CounterfactualLabPage.tsx # Side-by-side what-if matrix
└── src/App.tsx # Main UI with prompt & numeric input modes
```

PRESENTATION & EXAM PREP

Top 5 Professor / Viva Questions & Answers

Word-for-word answers to confidently explain your project

Frequently Asked Exam Questions

Q1: "Why did you use AO* instead of standard A* for this project?"

Your Answer: "Standard A* assumes simple serial transitions where one action leads to one state. But in autonomous aerial flight, taking a corridor requires decomposed subgoals and contingent constraints: reaching the next waypoint **AND** simultaneously clearing environmental safety checks (15% reserve battery, no prohibited airspace, no obstacles). AO* operates directly on **AND-OR graphs**, allowing conjunctive safety verification and automatic branch switching."

Q2: "So where does Rule-Based AI fit in if you are using AO*?"

Your Answer: "Our system is a **Hybrid AI architecture**. The Rule-Based AI acts as the deterministic safety auditor that defines the validity of the AND conjunct nodes. If a hard rule (like the 15% battery reserve floor) is violated, the rule engine marks that conjunct as FAIL with cost infinity. This failure immediately propagates up the AO* graph, causing AO* to prune that corridor and reroute to an alternative path."

Q3: "What is 'Counterfactual AI' and why is it important here?"

Your Answer: "Counterfactual AI means 'evaluating alternative what-if futures before acting.' Instead of blindly launching on a single brittle path, AERON runs 6 deterministic simulations pre-flight (e.g., What if battery drops to 34%? What if the destination vertipad closes?). This proves ahead of time whether the vehicle can safely handle adverse events."

Q4: "How do you guarantee that your heuristic is admissible?"

Your Answer: "An admissible heuristic must never overestimate remaining cost. We compute straight-line Euclidean distance to the destination assuming the vehicle travels at its maximum possible speed (60 m/s) with ideal minimal cruise power (15 kW). In any realistic flight, wind resistance, altitude changes, and turns will only increase time and energy, guaranteeing $h(n) \leq h^*(n)$."

Q5: "What happens if a disturbance happens while the vehicle is already in the air?"

Your Answer: "We implemented an In-Flight Dynamic Replanner. When an obstacle or battery drop is injected mid-flight, the system captures the vehicle's exact current waypoint and remaining battery state, re-runs AO* from that position, and smoothly splices the traversed path with the newly optimized trajectory without arbitrary teleportation."

🏆 **30-Second Elevator Pitch for Your Viva / Demo:**

"AERON is an autonomous aerial vehicle flight planner for urban eVTOL transport. We implemented a hybrid AI system combining ground-up AO heuristic search on AND-OR graphs with a deterministic rule-based safety layer. Unlike standard A* which fails during emergency reserve breaches, AERON evaluates simultaneous safety conjuncts, autonomously diverts to certified vertipads when battery is critical, and pre-simulates 6 counterfactual 'what-if' scenarios before vehicle dispatch."*